Code

```
#
╔═══════════════════════════════════════════════════════════════════════════════
═══════════════════════════╗
# ║          CANDY DISPENSER GAME  –  MicroPython / ESP32                ║
# ║                                                                      ║
# ║  MODE 1 – LUCK (Casino Slot Machine)                                 ║
# ║    Physical slot machine is spun by the player.                      ║
# ║    Host watches the result. If all 3 symbols match, host taps WIN    ║
# ║    in MIT App Inventor → BLE command → ESP32 runs all 3 servos       ║
# ║    together to dispense candy from every slot at once.               ║
# ║                                                                      ║
# ║  MODE 2 – SKILL (Colour Wheel Shuffle)                               ║
# ║    Player physically spins a 3-section colour wheel (R / G / B).     ║
# ║    A magnet on the arrow + 3 reed switches detect where it stops.    ║
# ║    The chosen colour lights up under all 3 dispenser slots.          ║
# ║    Player presses START. LEDs shuffle like a cup-shuffle game.       ║
# ║    All LEDs go dark. Player puts hand under the slot they think      ║
# ║    holds their colour. IR sensor detects the hand:                   ║
# ║      Correct → servo dispenses candy + NeoPixel win animation.       ║
# ║      Wrong   → buzzer sounds + red NeoPixel lose animation.          ║
#
╚═══════════════════════════════════════════════════════════════════════════════
═══════════════════════════╝

#
═══════════════════════════════════════════════════════════════════════════════
═══════════════════════════
#  PIN SUMMARY  (change these numbers to match YOUR actual wiring)
#
═══════════════════════════════════════════════════════════════════════════════
═══════════════════════════
#  Reed switches      GPIO 34, 35, 36    wheel sections  0=Red  1=Green  2=Blue
#  Start button       GPIO 0             active LOW  (internal pull-up enabled)
#  Mode switch        GPIO 4             HIGH = Luck mode  /  LOW = Skill mode
#  Servo signals      GPIO 25, 26, 27   one servo per dispenser (0, 1, 2)
#  IR sensors         GPIO 32, 33, 39   one under each dispenser (active LOW)
#  Buzzer             GPIO 23
#  NeoPixel data      GPIO 22
#  LEDs (9 total):
#    Dispenser 0 ->  Red=12   Green=13   Blue=14
#    Dispenser 1 ->  Red=15   Green=16   Blue=17
#    Dispenser 2 ->  Red=18   Green=19   Blue=21
#
═══════════════════════════════════════════════════════════════════════════════
═══════════════════════════
```

```
# Standard MicroPython libraries
import machine       # GPIO, PWM, hardware control
import time          # sleep, ticks_ms, ticks_diff
import random        # random.sample used to pick which slots swap during shuffle
import bluetooth     # built-in BLE stack on ESP32 MicroPython

# NeoPixel driver comes pre-installed with MicroPython on ESP32
from neopixel import NeoPixel


#
=========================================================================
=======
#  SECTION 1 – CONFIGURATION
#  All numbers you might want to tweak are gathered here at the top so you
#  never have to hunt through the code to change a pin or a timing value.
#
=========================================================================
=======

# GPIO pin numbers
REED_PINS  = [34, 35, 36]   # reed switches: index 0=Red, 1=Green, 2=Blue section
START_PIN  = 0              # start button (active LOW, internal pull-up)
MODE_PIN   = 4              # physical mode switch  HIGH=Luck / LOW=Skill

SERVO_PINS = [25, 26, 27]   # servo signal wires for dispensers 0, 1, 2
IR_PINS    = [32, 33, 39]   # IR obstacle sensors below dispensers 0, 1, 2
BUZZER_PIN = 23             # passive buzzer (needs PWM to make sound)

NEO_PIN    = 22             # NeoPixel data-in pin
NEO_COUNT  = 12             # total number of NeoPixel LEDs on your strip

# LED_PINS[dispenser][colour] gives the GPIO pin number
# dispenser 0-2 (left/middle/right slot), colour 0=Red 1=Green 2=Blue
LED_PINS = [
    [12, 13, 14],   # under dispenser 0
    [15, 16, 17],   # under dispenser 1
    [18, 19, 21],   # under dispenser 2
]

# Servo angles
SERVO_DISPENSE_ANGLE = 90   # rotate to this angle to open the candy gate
SERVO_REST_ANGLE     = 0    # return to this angle after dispensing (gate closed)

# Wheel-stop detection timings
SETTLE_MS = 400   # reed must stay LOW this long (ms) to count as truly stopped
SPIN_GAP  = 600   # all reeds must stay HIGH this long (ms) to confirm wheel spun
```

```
# Shuffle (cup-shuffle) speed tuning
SHUFFLE_ROUNDS  = 30    # total number of swap steps during the whole shuffle
SHUFFLE_FAST_MS = 55    # fastest LED swap interval at peak of shuffle (ms)
SHUFFLE_SLOW_MS = 280   # slowest interval at start/end of shuffle (ms)

# Other game timings
SHOW_COLOR_MS   = 2000   # how long the chosen colour is displayed before shuffle
HAND_TIMEOUT_MS = 12000  # how many ms the player has to put hand under a slot

# BLE device name shown in MIT App Inventor when scanning for devices
BLE_DEVICE_NAME = 'CandyGame'


#
==========================================================================
=======
#  SECTION 2 – COLOUR DEFINITIONS
#
==========================================================================
=======

# Human-readable names, index matches the reed switch index (0=Red etc.)
COLOR_NAMES = ['Red', 'Green', 'Blue']

# RGB tuples for the NeoPixel strip, one per colour index
NEO_COLORS = [
    (255,  40,   0),   # Red   (slightly warm orange-red looks better on WS2812)
    (  0, 220,  40),   # Green
    (  0,  60, 255),   # Blue
]

# Extra NeoPixel colours used in animations
NEO_OFF   = (0,   0,   0)    # all LEDs off
NEO_GOLD  = (255, 160,  0)   # gold jackpot glow used in Luck mode
NEO_WHITE = (180, 180, 180)  # soft white used in the chase animation
NEO_RED   = (200,   0,  0)   # red used for loss / error feedback


#
==========================================================================
=======
#  SECTION 3 – HARDWARE INITIALISATION
#
==========================================================================
=======

# Reed switches
# PULL_UP means the pin reads HIGH normally and only goes LOW when magnet is near
```

```
reed = [
    machine.Pin(p, machine.Pin.IN, machine.Pin.PULL_UP)
    for p in REED_PINS
]

# Start button: active LOW (pressed = LOW because pull-up holds it HIGH normally)
start_btn = machine.Pin(START_PIN, machine.Pin.IN, machine.Pin.PULL_UP)

# Mode switch: HIGH (switch open, pull-up wins) = Luck mode
#              LOW  (switch grounds the pin)    = Skill mode
mode_sw = machine.Pin(MODE_PIN, machine.Pin.IN, machine.Pin.PULL_UP)

# Servos
# Standard hobby servo uses 50 Hz PWM (20 ms period)
# Pulse width 0.5 ms = 0 deg,  1.5 ms = 90 deg,  2.5 ms = 180 deg
_servos = [
    machine.PWM(machine.Pin(p), freq=50)   # 50 Hz PWM on each servo pin
    for p in SERVO_PINS
]

def _angle_to_duty(angle):
    """
    Convert an angle (0 to 180 degrees) to a 10-bit PWM duty value (0 to 1023).

    Formula:
      pulse_ms = 0.5 + (angle / 180.0) * 2.0
        This gives:  0 deg -> 0.5 ms pulse
                    90 deg -> 1.5 ms pulse
                   180 deg -> 2.5 ms pulse

      duty = (pulse_ms / 20.0) * 1023
        Converts pulse width to a fraction of the 20 ms period,
        then scales to the 10-bit duty range the ESP32 expects.
    """
    pulse_ms = 0.5 + (angle / 180.0) * 2.0   # pulse width in milliseconds
    return int((pulse_ms / 20.0) * 1023)       # convert to 10-bit duty cycle

def servo_set(idx, angle):
    """Move servo number idx (0, 1, or 2) to the given angle (0 to 180 degrees)."""
    _servos[idx].duty(_angle_to_duty(angle))

def servo_all_rest():
    """Return all three servos to the closed rest position (gate shut)."""
    for i in range(3):
        servo_set(i, SERVO_REST_ANGLE)

def dispense_one(idx):
    """
```

```
    Open dispenser idx for one candy drop then close it again.
    600 ms open is usually enough for a single piece to fall through the gate.
    """
    servo_set(idx, SERVO_DISPENSE_ANGLE)  # rotate to open position
    time.sleep_ms(600)                    # hold open while candy falls
    servo_set(idx, SERVO_REST_ANGLE)      # rotate back to closed position
    time.sleep_ms(300)                    # brief settle before next action

def dispense_all():
    """
    Open all three servo gates simultaneously (Luck-mode jackpot).
    All gates open at the same time, then all close at the same time.
    """
    for i in range(3):
        servo_set(i, SERVO_DISPENSE_ANGLE)   # open all three at once
    time.sleep_ms(600)                       # wait for candy to fall
    for i in range(3):
        servo_set(i, SERVO_REST_ANGLE)       # close all three at once

# IR sensors
# These are active-LOW: output goes LOW when a hand (or any object) is nearby
ir = [
    machine.Pin(p, machine.Pin.IN)   # no pull-up needed, sensor has its own output
    for p in IR_PINS
]

# Buzzer
# A passive buzzer needs a PWM frequency to vibrate and produce sound.
# Setting duty=0 at startup keeps it silent until we want it.
_buz = machine.PWM(machine.Pin(BUZZER_PIN), freq=1000, duty=0)

def _beep(freq, ms):
    """
    Sound the buzzer at freq (Hz) for ms milliseconds, then silence it.
    duty=512 is 50% which gives maximum volume on a passive buzzer.
    The 30 ms gap at the end creates a clean separation between notes.
    """
    _buz.freq(freq)      # set pitch of the beep
    _buz.duty(512)       # turn on at 50% duty (loudest for passive buzzer)
    time.sleep_ms(ms)    # hold the note for this many milliseconds
    _buz.duty(0)         # silence the buzzer
    time.sleep_ms(30)    # short gap so consecutive notes sound distinct

def sound_win():
    """Happy ascending jingle played when the player wins."""
    for f in [523, 659, 784, 1047]:   # C5 E5 G5 C6: classic ascending win arpeggio
        _beep(f, 110)
```

```
def sound_lose():
    """Sad descending wah-wah tone played when the player loses."""
    for f in [400, 300, 200]:   # descending pitches for a deflating effect
        _beep(f, 200)

def sound_start():
    """Short double-beep confirming the START button was registered."""
    _beep(800, 80)
    _beep(1000, 80)

# 9 colour LEDs arranged in a 3x3 grid (3 dispensers x 3 colours)
# leds[dispenser][colour] is a Pin object configured as digital output
leds = [
    [machine.Pin(LED_PINS[d][c], machine.Pin.OUT) for c in range(3)]
    for d in range(3)
]

def leds_all_off():
    """Turn off every LED across all dispensers and all colours."""
    for d in range(3):
        for c in range(3):
            leds[d][c].off()

def led_on(dispenser, colour):
    """Turn on the LED at dispenser index d for colour index c (0=R 1=G 2=B)."""
    leds[dispenser][colour].on()

def led_off(dispenser, colour):
    """Turn off the LED at dispenser index d for colour index c."""
    leds[dispenser][colour].off()

def leds_show_arrangement(colours_at):
    """
    Light exactly one colour LED per dispenser slot according to the colours_at list.
    colours_at[d] = the colour index currently assigned to dispenser d.
    Clears all LEDs first to guarantee only one colour is lit per slot.
    """
    leds_all_off()          # clear everything first to avoid ghost lighting
    for d in range(3):
        led_on(d, colours_at[d])   # light the correct colour for this slot

# NeoPixel strip
# np[i] = (R, G, B) assigns a colour to pixel i
# np.write() pushes all pixel data to the physical strip at once
np = NeoPixel(machine.Pin(NEO_PIN), NEO_COUNT)

def np_fill(colour):
    """Set every NeoPixel to the same RGB colour tuple and push to strip."""
```

```
    for i in range(NEO_COUNT):
        np[i] = colour
    np.write()

def np_clear():
    """Turn off all NeoPixels by filling with black (0, 0, 0)."""
    np_fill(NEO_OFF)

def np_pulse(colour, times=4, on_ms=150, off_ms=100):
    """
    Flash the entire NeoPixel strip on and off a given number of times.
    Used for win celebrations and lose feedback to grab attention.
    on_ms  = how long each flash stays lit
    off_ms = gap between flashes
    """
    for _ in range(times):
        np_fill(colour)        # all pixels on with this colour
        time.sleep_ms(on_ms)
        np_clear()             # all pixels off
        time.sleep_ms(off_ms)

def np_chase(colour, cycles=2):
    """
    Knight-rider style chase animation.
    A bright pixel races around the strip with a dim ghost trailing behind it.
    Creates an exciting visual effect for win celebrations.
    cycles = how many full laps around the strip
    """
    for _ in range(cycles):
        for i in range(NEO_COUNT):
            np_clear()
            np[i] = colour                                  # bright lead pixel
            np[(i - 1) % NEO_COUNT] = tuple(v // 3 for v in colour)  # dim trailing ghost (1/3
brightness)
            np.write()
            time.sleep_ms(35)   # speed of the chase

def anim_win(colour_idx):
    """
    Full win celebration:
    1. Pulse the player's winning colour 3 times (their colour shines!)
    2. Pulse gold 3 times (jackpot feel)
    3. White chase animation around the strip (grand finale)
    """
    c = NEO_COLORS[colour_idx]             # get the RGB tuple for the winning colour
    np_pulse(c,        times=3, on_ms=120, off_ms=80)   # player's colour flashes
    np_pulse(NEO_GOLD, times=3, on_ms=120, off_ms=80)   # gold jackpot flashes
    np_chase(NEO_WHITE, cycles=2)                       # white chase as finale
```

```
    np_clear()

def anim_lose():
    """Rapid red pulsing animation to signal a wrong answer or no candy."""
    np_pulse(NEO_RED, times=6, on_ms=90, off_ms=70)
    np_clear()

def anim_startup():
    """
    Plays on boot-up to confirm all NeoPixels are working.
    Cycles through Red, Green, Blue then does a white chase.
    """
    for c in NEO_COLORS:
        np_fill(c)
        time.sleep_ms(250)
    np_chase(NEO_WHITE, 1)
    np_clear()


#
==========================================================================
=======
#  SECTION 4 – BLE UART  (Nordic UART Service)
#
#  This creates a standard BLE UART service that MIT App Inventor can connect
#  to using its BluetoothLE extension (no pairing required).
#
#  MIT App Inventor setup:
#    1. Add the BluetoothLE extension component to your app.
#    2. Call BluetoothLE.Connect using the device named "CandyGame".
#    3. Send commands using BluetoothLE.WriteStringValue with:
#        serviceUUID = "6E400001-B5B4-4455-8D65-B8843E86D541"
#        charUUID    = "6E400002-B5B4-4455-8D65-B8843E86D541"
#        value       = "WIN"  (or SKIP, LUCK, SKILL etc.)
#    4. Read responses by subscribing to notifications on:
#        charUUID    = "6E400003-B5B4-4455-8D65-B8843E86D541"
#
#  Commands the ESP32 understands (send from MIT App Inventor):
#    WIN  / JACKPOT / DISPENSE_ALL  -> all 3 servos fire (luck jackpot)
#    SKIP / LOSE / NO_WIN           -> lose animation, no candy
#    LUCK                           -> force Luck mode next round
#    SKILL                          -> force Skill mode next round
#
#  Responses sent back to the app:
#    WIN_DONE     -> jackpot animation finished
#    SKIP_DONE    -> skip animation finished
#    WIN:Red      -> player won with Red in skill mode
#    LOSE:Blue    -> player picked wrong, Blue was the correct colour
```

```
#    TIMEOUT      -> player did not choose in time
#
=========================================================================
=======

class BLEUART:
    # BLE event codes used to identify what happened in the IRQ callback
    _IRQ_CONNECT    = 1   # a phone/tablet connected to us
    _IRQ_DISCONNECT = 2   # the phone disconnected
    _IRQ_WRITE      = 3   # the phone sent data to us

    # Nordic UART Service UUIDs (industry-standard BLE UART profile)
    _SVC_UUID = bluetooth.UUID('6E400001-B5B4-4455-8D65-B8843E86D541')  # main
service
    _RX_UUID  = bluetooth.UUID('6E400002-B5B4-4455-8D65-B8843E86D541')  # phone
writes here
    _TX_UUID  = bluetooth.UUID('6E400003-B5B4-4455-8D65-B8843E86D541')  # ESP32
notifies here

    def __init__(self, name=BLE_DEVICE_NAME):
        self._ble   = bluetooth.BLE()      # get the BLE hardware object
        self._ble.active(True)             # power on the BLE radio
        self._ble.irq(self._irq)           # register our callback for BLE events
        self._conns = set()                # set of active connection handles
        self._rxbuf = bytearray()          # buffer to accumulate received bytes

        # Register the GATT service with TX (notify) and RX (write) characteristics
        # FLAG_NOTIFY  = ESP32 can push data to the phone without being asked
        # FLAG_WRITE   = phone can send data to ESP32
        TX_CHAR = (self._TX_UUID, bluetooth.FLAG_NOTIFY)
        RX_CHAR = (self._RX_UUID,
                   bluetooth.FLAG_WRITE | bluetooth.FLAG_WRITE_NO_RESPONSE)

        # gatts_register_services returns handles we store to read/write later
        ((self._tx_h, self._rx_h),) = self._ble.gatts_register_services(
            ((self._SVC_UUID, (TX_CHAR, RX_CHAR)),)
        )

        self._name = name.encode()   # encode device name to bytes for the ad payload
        self._advertise()            # start broadcasting so phones can discover us
        print(f"[BLE] Advertising as '{name}'")

    def _advertise(self):
        """
        Build a minimal BLE advertisement packet and begin broadcasting it.
        The payload contains two records:
          Flags record (3 bytes):  marks us as BLE-only, general discoverable
          Name record  (N+2 bytes): the human-readable device name
```

```
        gap_advertise interval is 100000 microseconds = every 100 ms.
        """
        payload = (bytes([0x02, 0x01, 0x06,          # Flags: LE discoverable, no BR/EDR
                          len(self._name) + 1, 0x09]) # Name header
                   + self._name)                      # Name bytes
        self._ble.gap_advertise(100_000, adv_data=payload)

    def _irq(self, event, data):
        """
        BLE interrupt handler called automatically by the MicroPython BLE stack.
        We handle three events:
          CONNECT    -> record the new connection handle
          DISCONNECT -> remove the handle, restart advertising so another device can
connect
          WRITE      -> phone sent us bytes; append to the receive buffer
        """
        if event == self._IRQ_CONNECT:
            conn_handle = data[0]              # unique number identifying this connection
            self._conns.add(conn_handle)       # add to our active connections set
            print(f"[BLE] Central connected (handle {conn_handle})")

        elif event == self._IRQ_DISCONNECT:
            conn_handle = data[0]
            self._conns.discard(conn_handle)   # remove from active connections
            print("[BLE] Central disconnected – re-advertising")
            self._advertise()                  # restart advertising for next connection

        elif event == self._IRQ_WRITE:
            # data[1] is the characteristic handle that received the write
            if data[1] == self._rx_h:          # confirm it was our RX characteristic
                # Read whatever bytes were written and append to our buffer
                self._rxbuf += self._ble.gatts_read(self._rx_h)

    def read_cmd(self):
        """
        Drain the receive buffer and return the latest command as a clean string.
        Strips leading/trailing whitespace and converts to uppercase so
        comparisons like cmd == 'WIN' always work regardless of phone formatting.
        Returns None if no data has been received yet.
        """
        if not self._rxbuf:
            return None                        # buffer is empty, nothing received
        raw = self._rxbuf.decode('utf-8', 'ignore').strip().upper()
        self._rxbuf = bytearray()              # clear the buffer after reading
        return raw or None                     # return None if string ended up empty

    def send(self, msg):
        """
```

```
    Send a text message back to the connected phone as a BLE notification.
    A newline character is appended so MIT App Inventor can detect message end.
    Silently ignores errors if the connection dropped mid-send.
    """
    data = (msg + '\n').encode()              # encode string to bytes
    for h in self._conns:
        try:
            self._ble.gatts_notify(h, self._tx_h, data)  # push to phone
        except Exception:
            pass   # connection may have dropped, ignore gracefully


#
=========================================================================
=======
#  SECTION 5 – LUCK MODE
#
=========================================================================
=======

def luck_mode(uart):
    """
    Luck mode: the physical casino slot machine is operated by the player.
    The HOST (game organiser) watches the physical slot machine result.
    If all 3 reels show the same symbol, the host taps WIN in the app.
    That BLE command fires all 3 servo dispensers simultaneously.

    The NeoPixels glow casino-gold while we wait for the host's decision.

    Accepted BLE commands:
      WIN / JACKPOT / DISPENSE_ALL  -> fire all 3 servos + win animation
      SKIP / LOSE / NO_WIN          -> lose animation, no candy dispensed
    """
    print("\n[LUCK] Luck Mode active. Waiting for host BLE command...")
    np_fill(NEO_GOLD)   # gold ambient glow sets the casino mood while waiting

    while True:
        cmd = uart.read_cmd()   # check if the MIT App Inventor app sent anything

        if cmd in ('WIN', 'JACKPOT', 'DISPENSE_ALL'):
            # Host confirmed jackpot: all 3 symbols matched on the slot machine
            print("[LUCK] JACKPOT! Dispensing candy from all 3 slots.")
            np_clear()
            dispense_all()          # open all three servo gates simultaneously
            sound_win()             # play ascending win jingle
            anim_win(0)             # gold + white NeoPixel win celebration
            uart.send('WIN_DONE')   # tell the app the sequence is complete
            time.sleep(2)           # brief pause before exiting
```

```
            np_clear()
            return   # end of this luck-mode round

        elif cmd in ('SKIP', 'LOSE', 'NO_WIN'):
            # Host confirmed the symbols did not all match: no candy
            print("[LUCK] No match. Better luck next time!")
            np_clear()
            sound_lose()
            anim_lose()
            uart.send('SKIP_DONE')
            return

        time.sleep_ms(50)   # small delay to avoid hammering the BLE stack in the loop


#
=========================================================================
=======
#  SECTION 6 – SKILL MODE HELPER FUNCTIONS
#
=========================================================================
=======

def skill_read_wheel():
    """
    Detect where the spinning colour wheel has STOPPED.

    THE PROBLEM WITH A SIMPLE REED CHECK:
    While the wheel is spinning, the magnet sweeps PAST each reed switch,
    briefly pulling it LOW then letting it go HIGH again as it moves on.
    If we just checked "is any reed LOW right now?" we would trigger on the
    very first sweep, before the wheel has stopped.

    THE SOLUTION – two-phase approach:

    PHASE 1: Confirm the wheel has been spun at all.
      We wait until ALL THREE reed switches read HIGH simultaneously for at
      least SPIN_GAP milliseconds. This means the magnet is not resting on any
      section and the wheel is freely moving. This also stops us from
      accidentally reading the leftover stopped position from the last round.

    PHASE 2: Detect a genuine sustained stop.
      We now watch for any reed to go LOW. The moment one does, we start a
      timer. If it stays LOW continuously for SETTLE_MS milliseconds without
      bouncing back HIGH, we know the wheel has truly stopped on that section
      (the magnet is resting there). If it goes HIGH again before SETTLE_MS
      we ignore it as a passing sweep and keep watching.
    """
```

```
print("[SKILL] Spin the wheel! Waiting for it to stop...")

# ── PHASE 1: wait for all reeds to be HIGH for SPIN_GAP ms ──────────────────
all_clear_start = time.ticks_ms()   # timestamp when all reeds were last all-HIGH

while True:
    if all(r.value() == 1 for r in reed):
        # All three reed switches are HIGH right now.
        # The magnet is not near any section at this instant.
        elapsed = time.ticks_diff(time.ticks_ms(), all_clear_start)
        if elapsed > SPIN_GAP:
            # All reeds have been consistently HIGH for over SPIN_GAP ms.
            # The wheel has been spinning freely. We can now look for a stop.
            break
    else:
        # At least one reed is LOW right now (magnet near a section).
        # Reset the clear-start timer because the wheel may be stationary.
        all_clear_start = time.ticks_ms()

    time.sleep_ms(10)   # poll every 10 ms to avoid burning CPU in a tight loop

# ── PHASE 2: detect a sustained LOW on one reed switch
──────────────────────────

while True:
    for idx, r in enumerate(reed):
        if r.value() == 0:
            # Reed idx just went LOW: the magnet is now near colour section idx.
            # Start timing to distinguish a passing sweep from a genuine stop.
            hold_start = time.ticks_ms()   # moment the LOW was first detected
            still_on   = True              # flag: stays True only if reed holds LOW

            # Keep checking this reed for SETTLE_MS milliseconds
            while time.ticks_diff(time.ticks_ms(), hold_start) < SETTLE_MS:
                if r.value() == 1:
                    # Reed went HIGH again BEFORE SETTLE_MS expired.
                    # The wheel was still moving and the magnet swept past.
                    # This is NOT a stop: set the flag and break the inner loop.
                    still_on = False
                    break
                time.sleep_ms(10)   # poll every 10 ms within the settle window

            if still_on:
                # The reed stayed LOW for the FULL SETTLE_MS uninterrupted.
                # The magnet is resting on this section. The wheel has stopped.
                print(f"[SKILL] Wheel stopped on -> {COLOR_NAMES[idx]}")
                return idx   # return 0=Red, 1=Green, or 2=Blue

    time.sleep_ms(10)   # outer poll: check all 3 reeds again if none triggered yet
```

```
def skill_show_chosen(colour):
    """
    After the wheel stops, illuminate the chosen colour LED in ALL THREE
    dispenser slots at the same time. This tells the player which colour
    they need to track through the upcoming shuffle.
    The NeoPixel strip also fills with the same colour for extra clarity.
    Everything turns off after SHOW_COLOR_MS so the shuffle can begin.
    """
    leds_all_off()                  # clear any LEDs left from a previous round
    for d in range(3):
        led_on(d, colour)           # same colour lit under every dispenser
    np_fill(NEO_COLORS[colour])     # NeoPixel matches the chosen colour strongly
    time.sleep_ms(SHOW_COLOR_MS)    # give the player time to memorise their colour
    np_clear()                      # turn NeoPixels off before the shuffle starts


def skill_shuffle(chosen):
    """
    Cup-shuffle animation: randomise which dispenser slot hides the player's
    chosen colour by swapping LED assignments between slots multiple times.

    INTERNAL TRACKING:
    colours_at is a list of 3 items where colours_at[d] = the colour index
    currently displayed (and logically assigned) to dispenser d.
    Initial state: dispenser 0 holds the chosen colour so the player sees it
    clearly before swapping starts.

    Each swap step picks 2 random dispensers and exchanges their colour
    assignments, then immediately updates the LEDs to show the new state.
    The player watches the moving LEDs to try to track their colour.

    SPEED CURVE (slow -> fast -> slow):
    The delay between swaps follows a bell-curve-like profile:
      Starts slow (player can follow easily)
      Accelerates to a peak confusion speed
      Slows back down (dramatic tension before going dark)
    This mirrors how real cup-shuffle artists move.

    At the end ALL LEDs go dark. The true colour layout is still recorded
    in colours_at which is returned so we can judge the player's answer.
    """
    # Build the starting arrangement: chosen colour is visible at slot 0
    colours_at = [
        chosen,             # slot 0 starts with the player's chosen colour
        (chosen + 1) % 3,   # slot 1 gets the next colour wrapping around 0-2
        (chosen + 2) % 3,   # slot 2 gets the remaining colour
```

```
    ]

    leds_show_arrangement(colours_at)   # display the starting arrangement
    time.sleep_ms(900)                  # let player lock eyes on their colour

    # Build the list of delays for each swap step (the speed curve)
    delays = []
    half = SHUFFLE_ROUNDS // 2   # the midpoint index where speed peaks

    for i in range(SHUFFLE_ROUNDS):
        if i < half:
            # First half: ramp from SLOW down toward FAST (delay decreases)
            # i/half goes from 0.0 to 1.0 as we approach peak speed
            t = SHUFFLE_SLOW_MS + (SHUFFLE_FAST_MS - SHUFFLE_SLOW_MS) * (i /
half)
        else:
            # Second half: ramp from FAST back up toward SLOW (delay increases)
            # (SHUFFLE_ROUNDS - i)/half goes from 1.0 down to 0.0
            t = SHUFFLE_SLOW_MS + (SHUFFLE_FAST_MS - SHUFFLE_SLOW_MS) *
((SHUFFLE_ROUNDS - i) / half)
        delays.append(int(t))   # append the integer millisecond delay

    # Execute each swap step in sequence
    for ms in delays:
        # Randomly pick 2 different dispenser indices to swap
        a, b = random.sample(range(3), 2)

        # Swap their colour assignments in the tracking list
        colours_at[a], colours_at[b] = colours_at[b], colours_at[a]

        # Update the physical LEDs to reflect the new arrangement
        leds_show_arrangement(colours_at)

        # Wait the ramp-controlled delay before doing the next swap
        time.sleep_ms(ms)

    # All LEDs off: the player must now rely on memory alone
    leds_all_off()
    np_clear()

    # Return the truth: what colour is actually at each dispenser right now
    return colours_at


def skill_wait_choice(chosen, colours_at, uart):
    """
    Wait for the player to slide their hand under one of the three dispenser
    slots. IR sensors (active LOW) detect the hand.
```

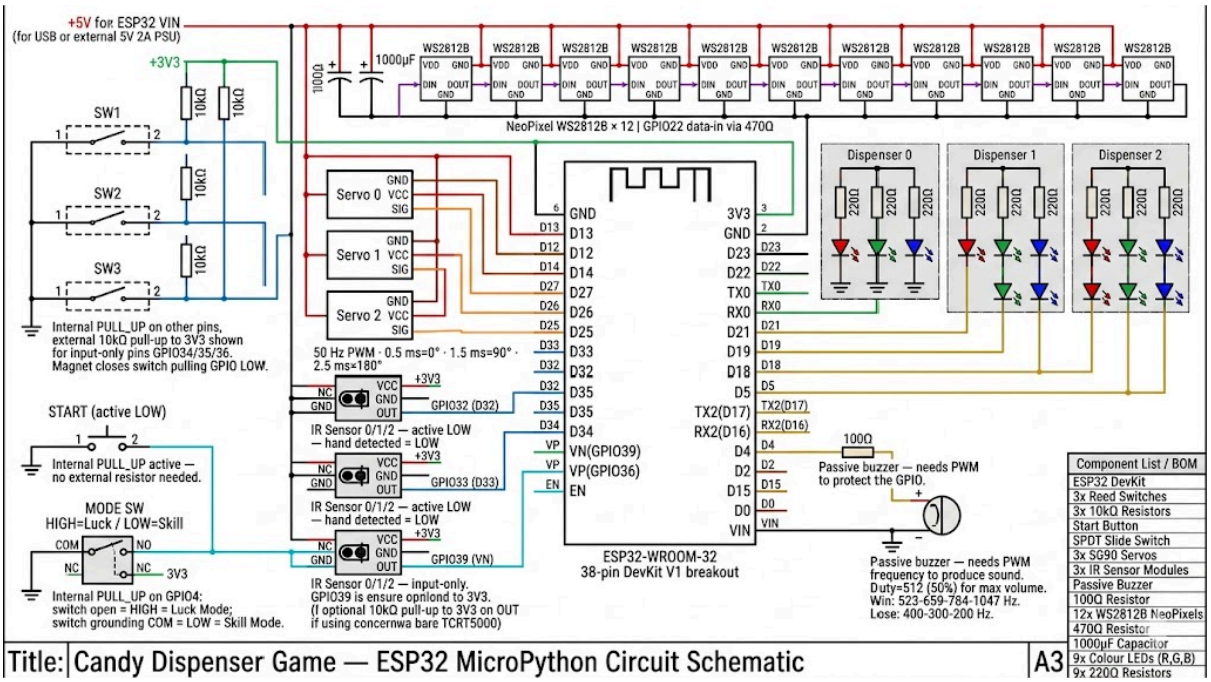
```
Parameters:
  chosen     - colour index the player was tracking (0=Red 1=Green 2=Blue)
  colours_at - list: colours_at[d] = actual colour at dispenser d post-shuffle
  uart       - BLEUART instance to send result back to the host app

Logic:
  Poll all 3 IR sensors until one goes LOW (hand detected) or timeout.
  Compare the colour at the chosen slot against the player's target.
  Correct: dispense candy + win sound + win NeoPixel animation.
  Wrong:   buzzer + red NeoPixel animation, no candy.
  Timeout: buzzer + red NeoPixel animation (player was too slow).
"""
print(f"[SKILL] Choose your slot! Track {COLOR_NAMES[chosen]}. "
      f"You have {HAND_TIMEOUT_MS // 1000} seconds.")

# Calculate the absolute deadline timestamp
deadline    = time.ticks_ms() + HAND_TIMEOUT_MS
picked_slot = -1   # -1 means no slot has been chosen yet

# Poll all IR sensors continuously until a hand is detected or we time out
while time.ticks_diff(deadline, time.ticks_ms()) > 0:
    for d in range(3):
        if ir[d].value() == 0:        # IR sensor output is LOW = hand nearby
            time.sleep_ms(40)         # debounce: wait 40 ms and re-check
            if ir[d].value() == 0:    # still LOW after debounce = confirmed hand
                picked_slot = d       # record which slot the player chose
                break                 # stop checking other sensors

    if picked_slot != -1:
        break   # a choice was made, exit the polling loop

    time.sleep_ms(10)   # pause 10 ms between polling cycles

# Evaluate the result
if picked_slot == -1:
    # The player never put their hand under any slot before time ran out
    print("[SKILL] TIMEOUT – no hand detected within time limit.")
    sound_lose()
    anim_lose()
    uart.send('TIMEOUT')
    return   # exit without dispensing candy

# Look up the actual colour that is at the slot the player chose
actual = colours_at[picked_slot]

if actual == chosen:
    # The slot the player chose really does contain their colour: CORRECT
```

```
        print(f"[SKILL] CORRECT! Slot {picked_slot} had "
              f"{COLOR_NAMES[chosen]}. Dispensing candy!")
        dispense_one(picked_slot)           # open only this servo to drop one candy
        sound_win()                         # play ascending win jingle
        anim_win(chosen)                    # NeoPixel win celebration
        uart.send(f'WIN:{COLOR_NAMES[chosen]}')   # notify host app

    else:
        # The slot the player chose had a different colour: WRONG
        print(f"[SKILL] WRONG! Slot {picked_slot} held "
              f"{COLOR_NAMES[actual]}, not {COLOR_NAMES[chosen]}. No candy.")
        sound_lose()
        anim_lose()
        uart.send(f'LOSE:{COLOR_NAMES[actual]}')  # tell app what the real colour was


def skill_mode(uart):
    """
    Runs one complete round of Skill mode in this order:
      1. Read wheel   - detect where it stopped to find the chosen colour
      2. Wait START   - player presses button when ready to play
      3. Show colour  - light chosen colour under all 3 slots for 2 seconds
      4. Shuffle      - swap LED colours between slots with speed ramp
      5. Wait choice  - player puts hand under their guessed slot
      6. Clean up     - turn off all LEDs and NeoPixels for next round
    """
    print("\n[SKILL] Skill Mode active.")
    leds_all_off()   # clear any LEDs from a previous round
    np_clear()

    # Step 1: Read the wheel to find which colour the player got
    chosen = skill_read_wheel()

    # Step 2: Wait for the player to press START
    print("[SKILL] Press the START button when ready!")
    # start_btn uses internal pull-up: HIGH = not pressed, LOW = pressed
    while start_btn.value() == 1:
        time.sleep_ms(50)    # poll every 50 ms to save power
    time.sleep_ms(150)       # debounce: ignore any bounce after the press
    sound_start()            # double-beep to confirm START was registered

    # Step 3: Show the chosen colour in all three slots
    skill_show_chosen(chosen)

    # Step 4: Run the cup-shuffle animation; returns true colour layout
    colours_at = skill_shuffle(chosen)

    # Step 5: Wait for the player to pick a slot and judge the answer
```

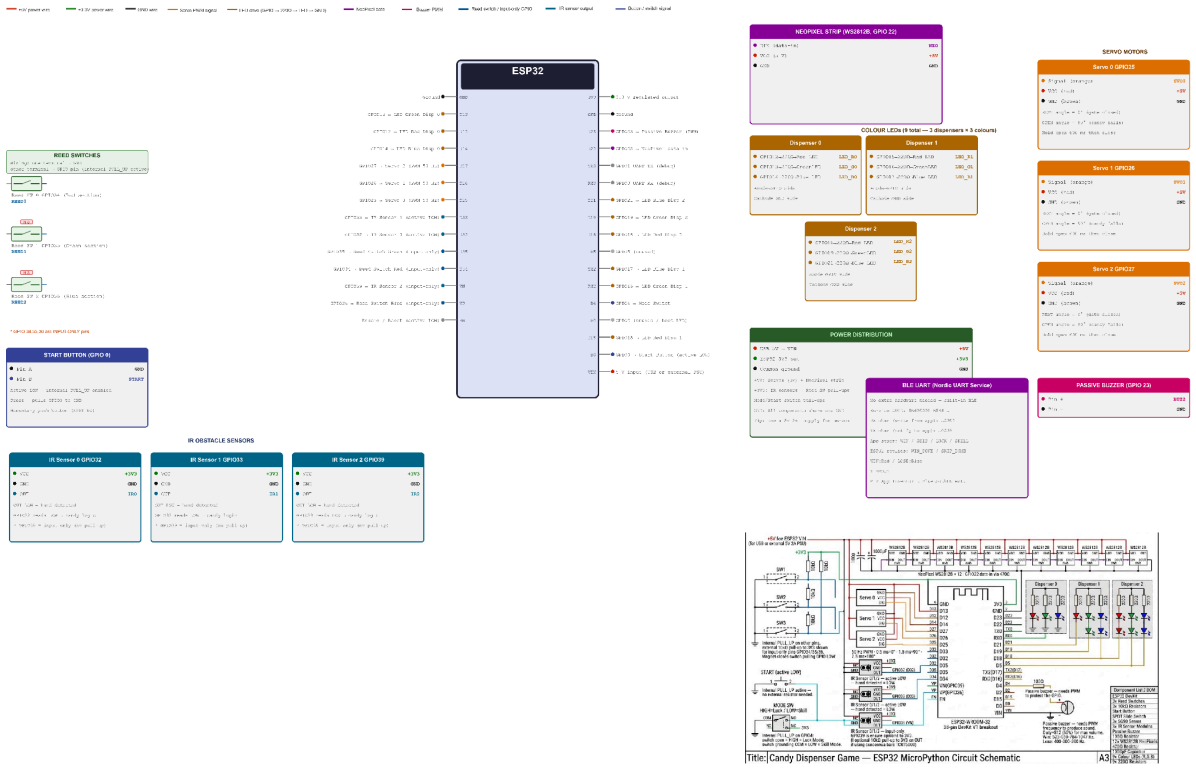
```
    skill_wait_choice(chosen, colours_at, uart)

    # Step 6: Clean up ready for the next round
    time.sleep(2)       # wait for animations to finish
    leds_all_off()
    np_clear()


#
==========================================================================
=======
#  SECTION 7 – MAIN LOOP
#
#  This is the top-level controller. It selects the game mode each round and
#  runs indefinitely. Mode selection priority:
#    1. BLE command (LUCK or SKILL from MIT App Inventor) - highest priority
#    2. Physical mode switch on GPIO 4 - used when no BLE command received
#
==========================================================================
=======

def main():
    print("╔═══════════════════════════════════════╗")
    print("╔═══════════════════════════════════════╗")
    print("╚═══════════════════════════════════════╝")

    # Start BLE UART so we can receive host commands from MIT App Inventor
    uart = BLEUART(BLE_DEVICE_NAME)

    # Put everything in a safe initial state
    servo_all_rest()   # all candy gates closed
    leds_all_off()     # all LEDs off

    # Play the startup animation to confirm all NeoPixels are working
    anim_startup()

    # Main game loop: runs forever, one round per iteration
    while True:
        # Check if the host app sent a BLE command this cycle
        cmd = uart.read_cmd()

        if cmd in ('WIN', 'JACKPOT', 'DISPENSE_ALL', 'SKIP', 'LOSE', 'NO_WIN'):
            # Host already sent an outcome command (not just a mode selection)
            # Treat this as a Luck mode round and handle the command immediately
            luck_mode(uart)

        elif cmd == 'LUCK':
            # Host selected Luck mode explicitly from the app
```

```
        print("[MAIN] BLE command: Luck mode selected")
        luck_mode(uart)

    elif cmd == 'SKILL':
        # Host selected Skill mode explicitly from the app
        print("[MAIN] BLE command: Skill mode selected")
        skill_mode(uart)

    else:
        # No BLE command received this cycle: use the physical mode switch
        # HIGH (switch open, pull-up holds it HIGH) = Luck mode
        # LOW  (switch shorts the pin to GND)       = Skill mode
        if mode_sw.value() == 1:
            luck_mode(uart)    # physical switch set to Luck side
        else:
            skill_mode(uart)   # physical switch set to Skill side

    # Short pause between rounds while servos and hardware settle
    time.sleep_ms(300)


# Entry point
# MicroPython automatically runs main.py on boot.
# All the code above defines functions and constants.
# This single call to main() is what actually starts the game.
main()
```

Circuit



Candy Dispenser Game — ESP32 Circuit Schematic



QUICK-REFERENCE (all nets at a glance)

GPIO 0	START	Start button active-LOW — 10kΩ pull-up to 3V3 (internal)	GPIO 18	LED_B2	LED Row Dispenser 2 — 220Ω → LED → GND	GPIO 32	IR0	IR sensor 0 OUT active-LOW 3V3-3V3 GPIO	
GPIO 4	BUZZ	Buzz output GPIO Buzz EFT Skill — Buzz-ON (LEDC/PWM)	GPIO 19	LED_G2	LED Green Dispenser 2 — 220Ω → LED → GND	GPIO 33	IR1	IR sensor 1 OUT active-LOW	
GPIO 12	LED_R0	LED Red Dispenser 0 — 220Ω → LED → GND	GPIO 21	SDA_GP	I²C SDA (Dispenser 0 — 220Ω → LED → GND)	GPIO 34	ADC0	Hall sensor ADC input (ADC1_CH6) — pull-up internal (Signal only pin)	
GPIO 13	LED_G0	LED Green Dispenser 0 — 220Ω → LED → GND	GPIO 22	SCL	Shared SCL (I²C — 4.7kΩ pull-up to 3V3 internal) (Dispenser 1 LED Strip)	GPIO 35	ADC1	Hall sensor ADC input (ADC1_CH7) — pull-up internal (Signal only pin)	
GPIO 14	LED_B0	LED Blue Dispenser 0 — 220Ω → LED → GND	GPIO 23	BUZZ	Passive buzzer output — pin 23 — GND	GPIO 36	ADC2	Hall sensor ADC — 220Ω pull-up (Internal (Signal only pin)	
GPIO 15	LED_R1	LED Red Dispenser 1 — 220Ω → LED → GND	GPIO 25	SERV	Servo 0 signal (50 Hz PWM) Drive-off and Jackpot-off / GND	GPIO 39	IR2	IR sensor 2 OUT active-LOW (Signal only pin)	
GPIO 16	LED_G1	LED Green Dispenser 1 — 220Ω → LED → GND	GPIO 26	SERV	Servo 1 signal (50 Hz PWM)				
GPIO 17	LED_B1	LED Blue Dispenser 1 — 220Ω → LED → GND	GPIO 27	SERV	Servo 2 signal (50 Hz PWM)				